

Name: Priyanshu Digari
Roll no. : 89

Assignment - 04 Neural Networks and Deep Learning

Q1. Case Study: Analyze CNN-based Image Classification System.

Ans. A Convolutional Neural Network (CNN) is widely used for image classification tasks such as object recognition, medical imaging, and facial recognition.

Case Study Example: Image Classification (e.g., Cats vs Dogs)

1. System Pipeline

Input: Raw images (e.g., $224 \times 224 \times 3$ RGB images)

Preprocessing:

Resize images

Normalize pixel values (0-1)

Data augmentation (flip, rotate)

2. CNN Architecture

Convolution Layers

Extract spatial features using filters

Activation Function

ReLU introduces non-linearity

Pooling Layers

max pooling reduces dimensions

Fully Connected Layers

Combine extracted features

Output Layer

Softmax for classification probabilities

3. Training

Loss Function: Cross-entropy

Optimizer: SGD / Adam

Backpropagation updates weights

4. Evaluation

Accuracy, Precision, Recall

Confusion matrix

5. Challenges

Overfitting → solved by dropout

Large dataset requirement

High computational cost

Q2. Numerical: Calculate Number of Parameters in DNN.

Ans. Formula:

For a fully connected layer:

Parameters = (Inputs $\times$ neurons) + biases

Example

Layer: 100 inputs $\rightarrow$ 50 neurons

$$(100 \times 50) + 50 = 5000 + 50 = 5050$$

multi-layer Example

Layer 1: 100 $\rightarrow$ 50 $\rightarrow$ 5050

Layer 2: 50 $\rightarrow$ 20 $\rightarrow (50 \times 20) + 20$
 $= 1020$

Output: 20 $\rightarrow$ 10 $\rightarrow (20 \times 10) + 10 =$
210

Total Parameters:

$$5050 + 1020 + 210 = 6280$$

Q3. Numerical: Output Size after Convolution and Pooling layers.

Ans. Convolution Formula

$$\text{Output size} = N - F + 2P/S + 1$$

Where:

N = input size

F = filter size

P = padding

S = stride

Example

Input = 32×32

Filter = 3

Padding = 1

Stride = 1

$$32 - 3 + 2(1) / 1 + 1 = 32$$

$$\rightarrow \text{Output} = 32 \times 32$$

Pooling Example

max Pool (2×2 , stride 2):

$$32 \rightarrow 16$$

$$\rightarrow \text{Final Output} = 16 \times 16$$

Q4. Explain RNN and its Variants.

Ans. Recurrent Neural Network (RNN)
Designed for sequential data (text, speech, time-series)

maintains hidden state (memory)

- Key idea:

$$h_t = f(Wx_t + Uh_{t-1})$$

- Limitations:

Vanishing gradient problem

Poor long-term memory

Variants of RNN

1. LSTM (Long Short-Term memory)

- Uses gates:

Forget gate

Input gate

Output gate

Handles long-term dependencies

2. GRU (Gated Recurrent Unit)

Simpler than LSTM

Combines gates

Faster training

3. Bidirectional RNN

Processes data forward + backward

Improves context understanding

4. Deep RNN

multiple stacked RNN layers

Q5. Training of Deep Networks Step-by-Step.

Ans. 1. Initialize Weights

Random initialization (Xavier/He)

2. Forward Propagation

Input passes through layers

Compute predicted output

3. Loss Calculation

Compare prediction with actual output

Example: Cross-entropy loss

4. Backpropagation

Compute gradients using chain rule

5. Weight Update

Update using optimizer:

$$W = W - \eta \cdot \nabla W$$

6. Iteration (Epochs)

Repeat until convergence

7. Regularization Techniques

Dropout